3.Operatori e espressioni

un espressione consiste in una sequenza di costanti , identificate , e gli operatori di un programma valuta a seconda delle operazioni indicate.

Un espressione si dice che ha un effetto collaterale se modifica qualche stato al di fuori del suo ambiente locale , questo vuol dire che ha un interazione con il "mondo esterno" e ritorna un valore

Ogni espressione ha un tipo, il tipo dell'espressione è il tipo del valore che risulta quando l'espressione viene valutata , se questa non contiene nessun valore è di tipo **void**

LVALUES e RVALUES

queste sono delle espressioni che definiscono un oggetto e per oggetto si intende alla locazione di memoria che rappresenta un determinato valore.
L'esempio più semplice e intuitivo è il nome che possiamo dare a una variabile come per esempio:

```
int pippo = 5;
double c = 12.2;
```

e da come possiamo notare gli LVALUES all'inizio hanno questa "L" che sta per left , quindi sinistra , perché in molti casi possono apparire a sinistra dell'operatore di assegnamento .

Tip

Spesso possono trovarsi anche a destra, come in questo caso

```
a = 7
b = a
```

Invece quelle espressioni che rappresentato dei valori senza designare un oggetto vengono chiamati come RVALUES,

Failure

gli RVALUES sono delle espressioni che possono apparire solamente a destra

Example

- 7 RVALUES è a = 7

  Failure

7 = a

- a LVALUE a= 7
- a +b rvalue a = a+b

> ✕ **Failure**
> a+b = 11

OPERATORS PRECEDENCE

un espressione può contenere diversi operatori , in questo caso le precedenze degli operatori determinano quale parte dell'espressione viene trattato come operando di ogni operatore.

Se due operatore in un espressione hanno la stessa precedenza ,la loro associatività determina se sono raggruppati con gli operandi in ordine da sx a dx o da dx a sx

Per esempio gli operatori aritmetici sono associati da sx a dx invece gli operatori di assegnazione da dx a sx

Gli operatori sono soggetti a usuali aritmetiche, il risultato di una divisione di 2 operatori integer sarà anche quello un integer

BITWISE OPERATORS

questa ti permette di manipolare bit individuali in un byte o in un larghi dati uniti:

- AND (&) = ris è 1 se e solo se entrambi i bit confrontati sono 1, se tutti altri casi è 0
- OR (|) = ris è 1 se almeno uno dei due bit è 1 , è 0 se entrambi è 0
- XOR(^) = ris è 1 se i bit sono diversi tra loto , se uguali ris 0
- NOT(~) = inverte tuti i bit

In memoria i valori vengono rappresentati in Little edian , nel processo questi vengono convertiti in bid edian , quindi le operazioni nel processo sono calcolato in big edian e quando il risultato viene archiviato ritorna nella memoria salvate in little edian

CONDITIONAL OPERATOR

Questi spesso vengono chiamati ternari o operatori ternari perche è l'unico operatore che ha 3 operatori

```
condizione ? espressione 1 : espressione 2
```

Le operazioni prima valutano la condizione successivamente, dipendentemente dal risultato , valuta l'altra o l'altra delle 2 presenti nell'espressione

```
int a= 6
int b= 12;
int c;
```

```
c= b > a ? 100: 150;
//c diventa 100
```

COMMA OPERATOR

l'operatore virgola è un operatore binario (espressione 1 , espressione 2), e assicura il processo sequenziale , dove valorizza il 1 valore , poi il 2 etc.
Il risultato d'essa deriva dalla completa espressine e ha come tipo il tipo del valore a destra dell'operatore .
L'operatore a sinistra è un valore valorizzato per i suoi effetti collaterali

SIDE EFFECTS, SEQUENCE POINTS

- Una funzione o espressione viene chiamata **side effect** , quando nell'addizione ritorna il valore e modifica alcuni stati o ha un interazione osservabile con le funzioni o con il mondo esterno
- A **sequence point** definisce qualsiasi punto in un programma eseguibile a cui è garantito che :
  - tutti gli effetti collaterali o le valutazione precedenti saranno stati eseguite
  - no sono ancora stati eseguiti effetti collaterali da successive valutazioni eseguite

Ricorda , che la modifica dell'operatore sinistra è un side effect di un espressione assegnata

```
a = b =5
```